

**FACULDADE DE TECNOLOGIA DO ESTADO DE SÃO PAULO ZONA SUL-
DOM PAULO EVARISTO ARNS**

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS-MANHÃ

RELATÓRIO DE PESQUISA

**INTRODUÇÃO A ALGORITMOS E LÓGICA DE
PROGRAMAÇÃO**

Unidade 1 – Conceitos Fundamentais

JOÃO VITOR RIBEIRO

SÃO PAULO

2026

JOÃO VITOR RIBEIRO

INTRODUÇÃO A ALGORITMOS E LÓGICA DE PROGRAMAÇÃO

Unidade 1 – Conceitos Fundamentais

Relatório de pesquisa elaborado a partir do material didático da aula “Introdução a Algoritmos e Lógica de Programação – Unidade 1: Conceitos Fundamentais”, complementado por referências acadêmicas e técnicas, com finalidade de documentação, revisão e apoio aos estudos.

SÃO PAULO

2026

RESUMO

Este relatório apresenta uma sistematização e ampliação do conteúdo introdutório sobre algoritmos e lógica de programação abordado na Unidade 1 da disciplina de Introdução a Algoritmos e Lógica de Programação. O material-base define algoritmo como uma sequência finita, ordenada e não ambígua de passos destinada à resolução de um problema ou à execução de uma tarefa, destacando ainda suas aplicações no cotidiano e as etapas que conduzem do problema ao programa: análise, elaboração do algoritmo, codificação, testes, depuração e manutenção. A pesquisa amplia esses conceitos por meio de referências acadêmicas e técnicas, discutindo a relação entre algoritmos, pensamento computacional, pseudocódigo, fluxogramas, linguagens de programação, testes e manutenção de software. O objetivo é oferecer uma explicação aprofundada, porém acessível, capaz de servir tanto como documentação acadêmica quanto como material de revisão para estudantes iniciantes em programação. Conclui-se que a programação não deve ser compreendida apenas como escrita de código, mas como um processo de resolução estruturada de problemas, no qual a qualidade do raciocínio lógico influencia diretamente a clareza, a correção, a testabilidade e a manutenção dos programas.

Palavras-chave: algoritmos; lógica de programação; pseudocódigo; fluxograma; programação; depuração; manutenção de software.

SUMÁRIO

1 INTRODUÇÃO

2 OBJETIVOS

3 METODOLOGIA

4 FUNDAMENTAÇÃO TEÓRICA

4.1 Conceito de algoritmo

4.2 Algoritmos no cotidiano

4.3 Características de um bom algoritmo

4.4 Do problema ao programa

4.5 Análise do problema

4.6 Elaboração do algoritmo

4.7 Codificação

4.8 Testes e depuração

4.9 Manutenção de programas

4.10 Por que estudar lógica de programação?

4.11 Ferramentas de apoio ao aprendizado

4.12 Erros comuns de iniciantes

5 EXEMPLO PRÁTICO INTEGRADOR

6 DISCUSSÃO

7 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

1 INTRODUÇÃO

O estudo de algoritmos constitui uma das bases da formação em Ciência da Computação e em cursos de desenvolvimento de software. Antes que um problema seja transformado em código, é necessário compreender o que se deseja resolver, identificar as informações disponíveis, determinar o resultado esperado e organizar uma sequência de ações que leve da entrada à saída. O material didático analisado apresenta exatamente essa perspectiva ao introduzir o aluno ao ciclo “problema–algoritmo–programa”, mostrando que o raciocínio lógico precede a escolha da linguagem de programação (MURAKAMI, s.d.).

A definição apresentada no material é simples e apropriada para iniciantes: algoritmo é uma sequência finita de passos ordenada e não ambígua capaz de resolver um problema ou executar uma tarefa, independentemente da linguagem utilizada (MURAKAMI, s.d.). Essa visão está alinhada à literatura introdutória de Ciência da Computação. O OpenStax, por exemplo, descreve algoritmo como uma sequência de instruções precisas que operam sobre dados e ressalta que um programa é uma implementação de um algoritmo em uma linguagem formal (OPENSTAX, 2024).

Essa distinção é importante porque programação e algoritmo não são sinônimos. O algoritmo representa a solução estruturada; o programa é a implementação dessa solução. Assim, duas pessoas podem escrever programas diferentes, em linguagens distintas, e ainda assim seguir essencialmente o mesmo algoritmo. Cormen et al. (2022) tratam os algoritmos como elementos centrais da computação e destacam a importância de projetá-los, descrevê-los e analisá-los de maneira sistemática.

O presente relatório parte do conteúdo dos slides da aula e o desenvolve de maneira didática. Além de explicar os conceitos principais, apresenta exemplos simples, relações com situações cotidianas e uma visão integrada do processo de desenvolvimento. A intenção não é substituir o material da disciplina, mas convertê-lo em uma documentação de estudo mais completa, útil para consulta posterior.

2 OBJETIVOS

2.1 Objetivo geral

Analisar e aprofundar os conceitos fundamentais de algoritmos e lógica de programação apresentados na Unidade 1 da disciplina, organizando-os em um material de revisão acadêmica, didático e tecnicamente fundamentado.

2.2 Objetivos específicos

- Explicar o conceito de algoritmo e suas principais características.
- Relacionar algoritmos a tarefas e situações do cotidiano.
- Apresentar o processo de transformação de um problema em programa.
- Explicar as formas de representação por linguagem natural, pseudocódigo e fluxograma.
- Diferenciar algoritmo de código e destacar a independência do raciocínio lógico em relação à linguagem.
- Compreender o papel dos testes, da depuração e da manutenção.
- Discutir a importância da lógica de programação para a formação de estudantes iniciantes.
- Identificar erros comuns durante o desenvolvimento de soluções computacionais.

3 METODOLOGIA

A elaboração deste relatório utilizou como fonte primária o material didático fornecido em PDF para a aula “Introdução a Algoritmos e Lógica de Programação – Unidade 1: Conceitos Fundamentais”, produzido pelo Prof. Dr. Lélis Tetsuo Murakami. O arquivo possui 19 páginas e aborda, entre outros pontos, o conceito de algoritmo, aplicações no cotidiano, características de bons algoritmos, análise do problema, elaboração, codificação, testes, depuração, manutenção, importância da lógica de programação, ferramentas e erros comuns de iniciantes (MURAKAMI, s.d.).

Como procedimento de aprofundamento, os tópicos dos slides foram confrontados e complementados com literatura acadêmica e documentação técnica. Foram utilizados como referências de apoio o livro “Introduction to Algorithms”, de Cormen et al., a obra “Computer Science: An Overview”, de Brooks e Brylow, materiais educacionais da Harvard CS50 e a documentação oficial da linguagem Python para aspectos ligados a ferramentas, depuração e ambiente de desenvolvimento. A pesquisa, portanto, combina análise documental e revisão bibliográfica, sem alterar a estrutura conceitual central apresentada na aula.

4 FUNDAMENTAÇÃO TEÓRICA

4.1 Conceito de algoritmo

O conceito de algoritmo pode ser entendido, em linguagem simples, como um conjunto de passos organizados para alcançar um objetivo. A palavra “organizados” é fundamental: não basta listar ações; elas precisam estar em uma ordem que faça sentido para o problema. Além disso, as instruções devem ser suficientemente claras para que não dependam de interpretações contraditórias. O material da aula destaca quatro ideias essenciais: sequência finita de passos, ordenação, ausência de ambiguidade e independência de uma linguagem de programação específica (MURAKAMI, s.d.).

A literatura de Ciência da Computação acrescenta que algoritmos operam sobre dados e devem oferecer um caminho concreto para transformar entradas em resultados. O OpenStax enfatiza que computadores executam instruções de maneira literal, o que explica por que a precisão é tão importante. Em um contexto humano, alguém pode corrigir mentalmente uma instrução vaga; em um sistema computacional, a ambiguidade normalmente resulta em comportamento incorreto ou em impossibilidade de execução (OPENSTAX, 2024).

Um exemplo simples é o cálculo da média de três notas. O objetivo é transformar três valores de entrada em um resultado. Uma forma de descrever o algoritmo seria: receber a primeira nota; receber a segunda nota; receber a terceira nota; somar as três; dividir a soma por três; apresentar a média. O algoritmo não é “Python”, “C” ou “Java”; ele é a lógica da solução. A linguagem será escolhida posteriormente, na etapa de codificação.

Exemplo didático de algoritmo: “receber três notas, calcular a soma, dividir por três e apresentar o resultado”.

A noção de algoritmo também envolve o término da execução. Uma sequência que não possui condição de encerramento pode entrar em repetição indefinida. Em atividades introdutórias, esse aspecto é especialmente relevante porque ajuda o aluno a perceber que uma solução computacional precisa contemplar não apenas o que fazer, mas também quando parar.

4.2 Algoritmos no cotidiano

O material utiliza exemplos cotidianos como receita culinária, manual de montagem de móveis, troca de pneu e trajeto até a faculdade para demonstrar que a ideia de algoritmo não nasceu com os computadores. Em todos esses casos existe uma meta e uma sequência de ações que contribuem para alcançá-la (MURAKAMI, s.d.).

A analogia com uma receita culinária é particularmente útil para iniciantes. Imagine que o objetivo seja preparar um café. É possível estabelecer uma sequência: separar os

ingredientes e utensílios; aquecer a água; colocar o filtro; adicionar o pó; despejar a água; aguardar a filtragem; servir. Se a ordem for alterada de maneira inadequada, o resultado pode não ser o esperado. A mesma ideia ocorre em algoritmos computacionais: algumas operações dependem de outras terem sido realizadas anteriormente.

Entretanto, existe uma diferença importante entre tarefas humanas e algoritmos destinados a computadores. Pessoas usam conhecimento de contexto para preencher lacunas. Um computador não possui automaticamente esse contexto. Por isso, quanto mais próximo o algoritmo estiver de uma linguagem formal, maior deve ser a preocupação com precisão, condições, entradas possíveis e resultados esperados.

4.3 Características de um bom algoritmo

Os slides apresentam quatro características centrais: finitude, definição ou não ambiguidade, entrada e saída e efetividade (MURAKAMI, s.d.). Essas características funcionam como critérios básicos para avaliar se uma solução é adequada.

4.3.1 Finitude

Um algoritmo deve possuir uma sequência que termine após um número finito de passos. Isso não significa que todos os algoritmos precisem ter um número pequeno de instruções, mas que deve existir uma condição de término para a execução considerada. Em programas reais, processos contínuos podem existir, como servidores que permanecem ativos, mas internamente suas estruturas de controle ainda são definidas por regras e condições específicas.

4.3.2 Definição e não ambiguidade

Cada passo precisa ser compreensível e interpretável de forma consistente. A instrução “faça rapidamente” é inadequada para um algoritmo porque “rapidamente” não define uma operação objetiva. Já “compare o valor A com o valor B e escolha o maior” é mais precisa. A precisão das instruções reduz erros de interpretação e facilita a transformação do algoritmo em código.

4.3.3 Entrada e saída

Entradas são as informações utilizadas pela solução. Saídas são os resultados produzidos. Nem todo algoritmo precisa receber dados do usuário em uma tela; entradas podem vir de arquivos, bancos de dados, sensores ou outras partes de um sistema. O importante é saber quais dados a solução utiliza e qual resultado deve produzir.

4.3.4 Efetividade

As etapas precisam ser executáveis e capazes de produzir progresso real na solução. Não adianta descrever uma ação impossível ou que não contribua para alcançar o objetivo. Em outras palavras, um bom algoritmo é conceitualmente correto, mas também precisa ser operacionalizável.

4.4 Do problema ao programa

Uma das ideias mais importantes da aula é o ciclo apresentado como “do problema ao programa”. O material organiza o processo em quatro grandes etapas: análise do problema, elaboração do algoritmo, codificação e testes, depuração e manutenção (MURAKAMI, s.d.). Esse fluxo representa uma visão introdutória, mas bastante útil, do desenvolvimento de software.

A separação entre essas etapas evita um erro comum de iniciantes: começar a escrever código antes de compreender o problema. A Harvard CS50 também estrutura seu ensino em torno de resolução de problemas, entradas e saídas, abstração, algoritmos e pseudocódigo, reforçando que a programação é mais ampla do que simplesmente conhecer a sintaxe de uma linguagem (MALAN, 2026).

Em um projeto simples, por exemplo, um aluno pode receber a tarefa de criar um programa que identifique o maior de três números. Antes de programar, deve definir quais são as entradas, qual é a saída, quais comparações são necessárias e como lidar com valores iguais. Só depois dessa definição a solução pode ser traduzida para uma linguagem.

4.5 Análise do problema

A análise do problema procura transformar uma solicitação vaga em uma especificação compreensível. O material pergunta: “O que é pedido, exatamente?”, “Quais dados de entrada existem?”, “Qual saída se espera?” e “Quais restrições ou casos especiais existem?” (MURAKAMI, s.d.). Essas perguntas são simples, mas representam uma das partes mais importantes do processo.

Considere a frase “faça um programa para calcular a média”. Há várias perguntas escondidas: média de quantos valores? As notas aceitam decimais? O resultado precisa ser arredondado? O que ocorre se não houver nenhuma nota? Quem fornecerá os dados? A média é aritmética? Uma análise cuidadosa transforma essas dúvidas em regras explícitas.

Esse estágio também é uma oportunidade para identificar casos de borda. Casos de borda são situações próximas dos limites esperados, como zero elementos, valores negativos,

notas iguais ao valor mínimo ou máximo, textos vazios e dados duplicados. Ignorá-los pode levar a um programa que funciona apenas no “caso feliz”, isto é, quando os dados são exatamente os que o programador imaginou.

4.6 Elaboração do algoritmo

Depois da análise, a solução pode ser representada de diferentes formas. Os slides destacam linguagem natural, pseudocódigo e fluxograma (MURAKAMI, s.d.). A escolha depende do objetivo e do grau de detalhamento desejado.

4.6.1 Linguagem natural

É a forma mais próxima da linguagem cotidiana. É útil para explicar a ideia geral da solução, especialmente quando o problema ainda está sendo compreendido. Seu ponto forte é a acessibilidade; sua limitação é que frases muito livres podem introduzir ambiguidades.

4.6.2 Pseudocódigo

Pseudocódigo é uma forma intermediária entre a linguagem natural e uma linguagem formal. Ele organiza a lógica com estruturas como “se”, “senão”, “enquanto”, “para”, entradas, saídas e atribuições, mas não precisa obedecer exatamente à sintaxe de uma linguagem de programação. A Harvard CS50 utiliza pseudocódigo como recurso para apresentar algoritmos sem depender da sintaxe específica de uma linguagem (HARVARD CS50, s.d.).

Pseudocódigo pode ser entendido como uma forma de escrever a lógica de uma solução de maneira estruturada sem assumir a sintaxe de uma linguagem específica.

4.6.3 Fluxograma

O fluxograma representa a lógica visualmente. Elementos gráficos podem indicar início e fim, processamento, entrada ou saída e decisões. Essa representação é útil quando o estudante precisa enxergar o caminho percorrido pelo algoritmo. O OpenStax observa que pseudocódigo e fluxogramas são formas usuais de representar algoritmos, permitindo que a lógica seja examinada antes da implementação (OPENSTAX, 2024).

Para iniciantes, uma comparação simples pode ajudar: o pseudocódigo funciona como uma receita escrita de maneira estruturada, enquanto o fluxograma funciona como um mapa visual dessa receita. Ambos podem representar a mesma solução.

4.7 Codificação

A codificação corresponde à tradução do algoritmo para uma linguagem de programação. O material destaca que a escolha da linguagem e sua sintaxe mudam, mas o

raciocínio lógico não precisa mudar (MURAKAMI, s.d.). Esse ponto é essencial para quem está começando: aprender uma nova linguagem não significa começar novamente do zero em relação à lógica de solução de problemas.

Por exemplo, um algoritmo que determina se uma pessoa é maior de idade possui uma regra lógica simples: receber a idade; verificar se a idade é maior ou igual ao limite adotado; informar o resultado. Essa mesma lógica pode ser implementada em Python, C, C#, JavaScript ou outra linguagem. O que muda é a forma de escrever os comandos.

A documentação oficial do Python apresenta a linguagem como simples de aprender e apropriada para diversas aplicações, além de disponibilizar tutorial, referência da linguagem e recursos de desenvolvimento (PYTHON SOFTWARE FOUNDATION, 2026). A escolha de Python nos exemplos da disciplina pode ser entendida, portanto, como uma estratégia didática para concentrar a atenção nos conceitos fundamentais sem exigir, inicialmente, uma sintaxe excessivamente complexa.

4.8 Testes e depuração

A etapa de testes procura verificar se o comportamento do programa corresponde ao resultado esperado. O material orienta executar o sistema com diferentes entradas, comparar a saída obtida com a saída esperada, identificar erros de lógica, corrigir e testar novamente (MURAKAMI, s.d.).

Testar não significa apenas verificar se o programa “abre”. Um programa pode executar sem gerar uma mensagem de erro e, ainda assim, produzir resultados incorretos. Por isso, é importante criar casos de teste que cubram situações normais, valores-limite, valores inesperados e combinações relevantes de entradas.

Depuração, ou debugging, é o processo de investigar e corrigir o comportamento incorreto. A documentação oficial do Python mostra que ferramentas de depuração permitem percorrer a execução passo a passo, usar pontos de parada e inspecionar o estado do programa (PYTHON SOFTWARE FOUNDATION, 2026). Isso demonstra que a depuração não é uma atividade improvisada, mas uma prática sistemática de desenvolvimento.

Um exemplo simples é um programa que calcula a média de duas notas. Se o programador dividir a soma por três por engano, o programa provavelmente executará normalmente, mas dará a resposta errada. O erro é lógico, não necessariamente sintático. Esse tipo de problema reforça por que testar a saída esperada é indispensável.

4.9 Manutenção de programas

A manutenção aparece no material como uma etapa posterior, porém permanente. Programas evoluem com o tempo; falhas podem ser descobertas depois da entrega; necessidades de usuários podem mudar; novas regras de negócio podem surgir. Por isso, o software não deve ser visto como um artefato terminado para sempre (MURAKAMI, s.d.).

O cuidado com a organização do código é particularmente importante nesse contexto. Um código mais claro facilita a localização de erros e a realização de alterações futuras. Embora o primeiro exercício de programação possa parecer pequeno, o hábito de organizar soluções desde o começo cria uma base para projetos mais complexos.

É possível distinguir, de maneira didática, três situações de manutenção: corretiva, quando o objetivo é corrigir falhas; adaptativa, quando o software precisa acompanhar mudanças no ambiente; e evolutiva, quando novas funcionalidades ou melhorias são incorporadas. Essas categorias ajudam a entender por que sistemas reais exigem trabalho contínuo depois da primeira versão.

4.10 Por que estudar lógica de programação?

O material apresenta quatro motivos principais: a lógica serve de base para qualquer linguagem, desenvolve raciocínio estruturado, é aplicável fora da programação e reduz a curva de aprendizado futura (MURAKAMI, s.d.). Esses argumentos encontram eco em materiais introdutórios de Ciência da Computação, que apresentam programação como uma prática de resolução de problemas e não apenas como domínio de uma sintaxe.

A lógica de programação também ajuda a decompor problemas grandes em partes menores. Essa habilidade é conhecida como decomposição e está associada ao pensamento computacional. Em vez de tentar resolver tudo ao mesmo tempo, o estudante identifica pequenas tarefas, define suas relações e testa cada parte. A partir daí, fica mais fácil construir uma solução maior.

Outro benefício é a transferência para atividades não relacionadas diretamente ao código. Organizar etapas, definir prioridades, trabalhar com condições e prever consequências são formas de raciocínio que podem ser úteis em planejamento de estudos, análise de processos e resolução de problemas cotidianos. A própria abordagem da CS50 enfatiza que as competências de resolução de problemas trabalhadas no curso ultrapassam a programação em si (MALAN, 2026).

4.11 Ferramentas de apoio ao aprendizado

Os slides indicam o uso de pseudocódigo, fluxogramas, linguagem de alto nível, com exemplos em Python, e um ambiente de desenvolvimento para prática (MURAKAMI, s.d.). Cada ferramenta tem uma função pedagógica distinta.

O pseudocódigo permite testar a lógica antes de preocupar-se excessivamente com a sintaxe. O fluxograma fornece uma visão visual. A linguagem de alto nível possibilita transformar o raciocínio em um programa executável. Já o ambiente de desenvolvimento reúne editor, execução e recursos de depuração, formando o espaço no qual o aluno experimenta, testa e corrige soluções.

Na documentação do Python, ambientes e editores podem oferecer destaque de sintaxe, ferramentas de debugging e verificações relacionadas ao estilo do código. Isso mostra que ferramentas não substituem a lógica; elas apoiam o estudante durante a construção, execução e investigação da solução (PYTHON SOFTWARE FOUNDATION, 2026).

4.12 Erros comuns de iniciantes

A aula identifica quatro erros recorrentes: pular diretamente para o código, confundir atribuição com igualdade, não testar casos de borda e não revisar a lógica antes de codificar (MURAKAMI, s.d.). Esses erros são comuns porque o iniciante tende a associar programação à escrita imediata de comandos.

Pular diretamente para o código pode gerar retrabalho. Sem uma definição clara da solução, o estudante pode escrever várias linhas e só depois perceber que interpretou o problema de maneira incorreta. A elaboração prévia do algoritmo funciona como uma etapa de planejamento que reduz esse risco.

A confusão entre atribuição e igualdade é outro exemplo. Em muitas linguagens, o símbolo utilizado para atribuir um valor a uma variável é diferente do símbolo usado para comparar valores. Como essa convenção varia entre linguagens, o estudante precisa compreender o conceito lógico por trás da operação, e não apenas memorizar um símbolo.

Não testar casos de borda também é perigoso. Um programa pode funcionar para entradas comuns e falhar justamente nas situações que mais exigem cuidado. Testar valores mínimos, máximos, zero, repetição e entradas inesperadas é parte do desenvolvimento da solução.

Finalmente, revisar a lógica antes da codificação permite encontrar inconsistências quando ainda são simples de corrigir. Depois que um erro é incorporado ao código, sua correção pode exigir mais esforço porque pode afetar outras partes do programa.

5 EXEMPLO PRÁTICO INTEGRADOR

Para reunir os conceitos apresentados, considere o seguinte problema: “Desenvolver um programa que receba três notas de um estudante, calcule a média e informe se o estudante foi aprovado, considerando média mínima de 6,0”. O problema pode ser resolvido seguindo o ciclo apresentado na aula.

5.1 Análise

Entradas: três notas numéricas. Saídas: média calculada e situação do estudante. Regra: média maior ou igual a 6,0 significa aprovação; caso contrário, reprovação. Casos especiais: verificar se as notas estão dentro de uma faixa válida, caso a especificação do sistema exija isso.

5.2 Algoritmo em linguagem estruturada

1. Receber a primeira nota.
2. Receber a segunda nota.
3. Receber a terceira nota.
4. Somar as três notas.
5. Dividir a soma por 3.
6. Se a média for maior ou igual a 6,0, informar “Aprovado”.
7. Caso contrário, informar “Reprovado”.

5.3 Pseudocódigo

```
INÍCIO
  Leia          nota1
  Leia          nota2
  Leia          nota3
  media ← (nota1 + nota2 + nota3) / 3
  Se media >= 6 então
    Escreva      “Aprovado”
  Senão
    Escreva      “Reprovado”
  FimSe
FIM
```

5.4 Codificação conceitual

A etapa seguinte seria traduzir o pseudocódigo para uma linguagem de programação. Em Python, por exemplo, a sintaxe mudaria, mas a sequência lógica permaneceria a mesma: entrada dos dados, cálculo da média, comparação e saída. Esse exemplo demonstra, na prática, a afirmação dos slides de que a sintaxe muda entre linguagens, mas o raciocínio lógico não precisa mudar (MURAKAMI, s.d.).

5.5 Testes

Um conjunto inicial de testes poderia utilizar notas 6,0; 6,0; 6,0, esperando média 6,0 e aprovação. Outro teste poderia usar 5,0; 5,0; 5,0, esperando média 5,0 e reprovação. Um terceiro caso poderia testar uma situação próxima do limite, como 5,9; 6,0; 6,1. O objetivo é verificar se a condição foi implementada corretamente.

5.6 Depuração

Suponha que o programa informe “Aprovado” para média 5,0. Nesse caso, a comparação deve ser investigada. O programador pode verificar o valor da variável *media* durante a execução, conferir a expressão de cálculo e testar novamente. Assim, a depuração transforma um resultado inesperado em uma sequência de verificações que permite localizar a causa do erro.

6 DISCUSSÃO

A análise do material demonstra que os conteúdos selecionados para a Unidade 1 formam uma sequência didática coerente para estudantes que estão iniciando na programação. O professor começa pelo conceito de algoritmo, aproxima a ideia do cotidiano, apresenta critérios de qualidade e, em seguida, conduz o aluno ao processo de transformar um problema em programa. Essa organização é consistente com materiais introdutórios que apresentam primeiro resolução de problemas, algoritmos, pseudocódigo e somente depois detalhes mais específicos da programação.

Um aspecto pedagogicamente relevante é a insistência em não confundir o algoritmo com a linguagem. Para o iniciante, existe uma tendência natural de acreditar que “saber programar” significa memorizar comandos. Contudo, a literatura de algoritmos mostra que o objeto principal é a solução do problema. Cormen et al. (2022) apresentam algoritmos em pseudocódigo justamente para enfatizar a lógica e permitir sua compreensão sem depender da implementação final.

Outro ponto forte da aula é a presença de testes e manutenção já no primeiro contato com programação. Isso amplia a visão do estudante. Um programa não termina quando compila: ele precisa ser testado, corrigido e posteriormente mantido. A documentação oficial de ferramentas de depuração reforça que investigar a execução faz parte do trabalho técnico do programador (PYTHON SOFTWARE FOUNDATION, 2026).

Do ponto de vista formativo, a principal mensagem da Unidade 1 pode ser resumida da seguinte maneira: programar é resolver problemas de maneira sistemática. A linguagem escolhida é importante, mas ela representa a etapa de implementação. Antes dela vêm a compreensão do problema, a definição das entradas e saídas, a construção da lógica e a verificação da solução.

7 CONSIDERAÇÕES FINAIS

O conteúdo estudado mostra que algoritmos e lógica de programação constituem fundamentos indispensáveis para a formação de profissionais de tecnologia. Um algoritmo é uma sequência finita e não ambígua de instruções organizada para atingir um objetivo. Ele pode ser representado por linguagem natural, pseudocódigo ou fluxograma e, posteriormente, transformado em código por meio de uma linguagem de programação (MURAKAMI, s.d.; OPENSTAX, 2024).

O processo completo não termina com a codificação. A análise correta do problema ajuda a definir entradas, saídas, restrições e casos especiais. A elaboração do algoritmo permite estruturar a solução. A codificação transforma essa solução em software. Os testes verificam o comportamento, a depuração investiga e corrige falhas e a manutenção acompanha a evolução do programa.

Para o estudante iniciante, a principal conclusão é que aprender programação não significa apenas decorar comandos. É necessário aprender a pensar em etapas, decompor problemas, identificar regras, testar hipóteses e revisar soluções. Esse conjunto de habilidades forma a base para o aprendizado posterior de estruturas de controle, funções, estruturas de dados, bancos de dados e outras áreas da computação.

Dessa forma, a Unidade 1 funciona como uma base conceitual para as próximas etapas da disciplina. Ao consolidar o ciclo problema–algoritmo–programa, o estudante passa a enxergar o código não como um conjunto aleatório de instruções, mas como a implementação de uma solução previamente raciocinada.

REFERÊNCIAS

BROOKSHEAR, J. Glenn; BRYLOW, Dennis. Computer Science: An Overview. 13. ed. Boston: Pearson, 2021.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022.

HARVARD CS50. Week 0 – Scratch / Algorithms and pseudocode. Harvard University, 2026. Disponível em: <https://cs50.harvard.edu/x/weeks/0/>. Acesso em: 15 ago. 2026.

MALAN, David J. CS50 AP – Lecture 0. Harvard University, 2026. Disponível em: <https://cs50.harvard.edu/ap/2026/curriculum/x/notes/0/>. Acesso em: 15 ago. 2026.

MURAKAMI, Lélis Tetsuo. Introdução a Algoritmos e Lógica de Programação: Unidade 1 – Conceitos Fundamentais. Material didático em PDF. s.l.: s.n., s.d.

OPENSTAX. Introduction to Computer Science: 2.1 Computational Thinking; 3.1 Introduction to Data Structures and Algorithms. Houston: OpenStax, 2024. Disponível em: <https://openstax.org/>. Acesso em: 15 ago. 2026.

PYTHON SOFTWARE FOUNDATION. Python 3 Documentation. 2026. Disponível em: <https://docs.python.org/3/>. Acesso em: 15 ago. 2026.

PYTHON SOFTWARE FOUNDATION. Debugging and Profiling. Python 3 Documentation, 2026. Disponível em: <https://docs.python.org/3/library/debug.html>. Acesso em: 15 ago. 2026.